

Team 7 - Meeting Minutes

Project: Web-Based Language Scavenger Hunt Game

Meeting Date: Tuesday, September 29

Next Meeting: Sunday, October 4

Attendees: Gamuchirai Mhere, Madiba Burks Magara, Ashish Nayak, Dan Rukundo

Agenda

Setting up the initial dev environment and project infrastructure ahead of Milestone 1 work.

Action Items — Due Sunday, October 4

#	Task	Assignee	Details
1	Set up GitHub Actions	Gamu	Configure CI/CD pipeline to run automatically on every push/PR to <code>main</code> : run PyTest (backend) and Vitest (frontend), add a linting step (flake8/black for Python, ESLint for TS/React), fail the build on test/lint failure, and confirm a status badge shows on the repo README.
2	Set up FastAPI	Ashish	Scaffold the base FastAPI backend: initialize project with a basic <code>/health</code> or <code>/ping</code> endpoint, set up folder structure (routers, models, schemas, services), add <code>requirements.txt</code> /Poetry with core dependencies, and confirm it runs locally.
3	Set up Docker	Dan	Containerize the backend and database for consistent local dev: write a <code>Dockerfile</code> for the FastAPI app, write a <code>docker-compose.yml</code> including the app + PostgreSQL service, confirm <code>docker-compose up</code> runs the API and connects to the DB, and document setup steps in the README.
4	Add M1 requirements to Kanban	All	Break down the Milestone 1 functional requirements table into individual cards: one card per requirement (not per test case), include the requirement text plus its positive/negative test cases, assign per the proposal's Member column, and tag with an M1 label.
5	Set up PostgreSQL	Madiba	Stand up the project database: create the initial schema (users, roles, classes, class-membership tables per Table 1), set environment variables/connection config for local + Docker use, confirm FastAPI can connect and query it, and document schema decisions in the repo.

Notes

- All five tasks are due by Sunday, October 4.
- These form the foundation for Milestone 1 (Months 1–4): the core data model comes first, since all later backend work depends on it.